# Audit Report

## Own Protocol v2

Date: **November 7 — November 14, 2026**

# Introduction

eUSD is an overcollateralized stablecoin issued against tokenized real-world assets. Each collateral risk group has its own manager contract, and all managers mint the same eUSD. This review covers the EUSDLiquidityManager, which mints eUSD against concentrated-liquidity positions in asset/eUSD pools. Each position lives in its own vault (a beacon proxy of a DEX adapter: Fables, a Uniswap v4 fork, or Aerodrome Slipstream). The manager values a position at the oracle price, which is a lower bound for a concentrated-liquidity range at any pool price. It charges a stability fee that depends on whether the liquidity is staked in the DEX gauge. Positions can be liquidated in part or in full. Any eUSD holder can redeem eUSD against positions whose vault is mostly asset at the pool price.

The review then covered two later changes. The first lets a position owner name a delegate, such as an LP agent, for their vault. `rebalance` replaces `reposition`: it moves the range and can swap inside the vault, and a per-vault loss budget bounds what the delegate's calls can lose. Vaults can also harvest DEX emissions. The second adds the EUSDPsm, an upgradeable peg stability module that mints eUSD one for one against listed stablecoins (each with its own debt ceiling, fees and mint pause) and redeems eUSD back, and PegArbitrage, a keeper helper that trades eUSD pools on Uniswap v4 forks (such as Fables), Uniswap v3 and Aerodrome Slipstream back towards $1 through the PSM, without holding eUSD.

The review ran in rounds. Each round used independent multi-agent passes over the source. Every finding was re-verified against the code and, for the DEX adapters, against the live Robinhood Chain contracts. Fixes were re-audited. Proofs of concept ran on a Robinhood Chain fork or as unit tests.

Two final full reviews of all EUSDLiquidityManager files ran independently on aba9e7e, after the last fixes. Neither raised a new issue. Every point they raised was already in this report as acknowledged (for example I-06 on partial liquidation below 100%), or was checked earlier against the live Robinhood Chain contracts and rejected. The delegate changes were re-audited after each round of fixes, and the final review of ae2a1f5 raised no new issue. The PSM and PegArbitrage findings are all acknowledged; a final full review of af579b6 raised no new finding.

# Scope

Repository: `own-protocol/own-v2`

Liquidity Manager Audited Commit: `c4042f53315915478673b399bf3b3c697149c8ff`

Liquidity Manager Re-audit Commit: `aba9e7ea36c551e744a2422039cd1463b32e1f2f`

LP Agent Audited Commit: `bda4e6c5d67c5859472376e674746a39a43cc438`

LP Agent Re-audit Commit: `ae2a1f5f5abbbda72f378c3f2201e8bf22e8ef64`

PSM Audited Commit: `97b55392d94c3c12b106078aa5b940bc27f5dc96`

PSM Re-audit Commit: `af579b64d395ad7f1985a6e5d03128061ce3554b`

Files:

- `src/eusd/EUSDLiquidityManager.sol`
- `src/eusd/adapters/FablesAdapter.sol`
- `src/eusd/adapters/AerodromeAdapter.sol`
- `src/libraries/LiquidityMath.sol`
- `src/libraries/DelegateLoss.sol`
- `src/interfaces/IEUSDLiquidityManager.sol`
- `src/interfaces/ILiquidityVault.sol`
- `src/interfaces/external/IFables.sol`
- `src/interfaces/external/IAerodrome.sol`
- `src/eusd/EUSDPsm.sol`
- `src/interfaces/IEUSDPsm.sol`
- `misc/PegArbitrage.sol`
- `misc/interfaces/IPegArbitrage.sol`
- `misc/interfaces/IPoolManagerSwap.sol`
- `misc/interfaces/IUniswapV3PoolSwap.sol`

The EUSDLiquidityManager and its adapters are deployed on Robinhood Chain from the LP agent re-audit commit; addresses are in `docs/contracts-robinhood.md`. The

EUSDPsm and PegArbitrage are reviewed at the PSM re-audit commit. The oracle (`ChainlinkOracleVerifier`), `EUSD` and `EUSDManager` were read as context and are out of scope.

# Findings

Each issue has an assigned severity:

**Critical/High** issues are directly exploitable security vulnerabilities that can lead to loss of funds and need to be fixed.

**Medium** issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.

**Low/Info** issues are non-exploitable or informational findings that do not pose a direct security risk to the system's integrity.

Issues marked `RESOLVED` are fixed in the codebase and carry a regression test that fails without the fix. Issues marked `ACKNOWLEDGED` are accepted by design, with the rationale and any operational mitigations documented.

## Issues Found

| Critical | High | Medium | Low | Info |
|---|---|---|---|---|
| 0 | 2 | 7 | 9 | 15 |

## Issues Not Fixed and Not Acknowledged

| Critical | High | Medium | Low | Info |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |

## Summary of Findings

| ID | Severity | Title | Status |
|---|---|---|---|
| **H-01** | High | Donated liquidity on an unstaked Aerodrome vault NFT makes the whole debt unliquidatable | Fixed |
| **H-02** | High | `rebalance` checks the pool price band before its own swap, so the new range can open at a price the swap moved | Fixed |
| **M-01** | Medium | A partial liquidation can take more than its share of the collateral and leave bad debt | Fixed |
| **M-02** | Medium | Redemption pays the oracle price with no buffer, so redeemers keep the oracle's lag behind the market | Fixed |
| **M-03** | Medium | A delegate can repeat `rebalance` with no limit on the owner's total loss | Fixed |
| **M-04** | Medium | `liquidate` counts harvested idle eUSD on both sides, so a healthy position can be liquidated | Fixed |
| **M-05** | Medium | The OPERATOR can send the PegArbitrage float to a pool it controls | Acknowledged |
| **M-06** | Medium | eUSD minted with a depegged stablecoin redeems a sound one | Acknowledged |
| **M-07** | Medium | A zero-fee PSM mint funds EUSDManager redemptions at a lagging oracle price | Acknowledged |
| **L-01** | Low | A liquidator can move the pool price to earn the bonus on the owner's eUSD | Fixed |
| **L-02** | Low | A blocklisted owner makes the final full liquidation revert | Acknowledged |

| | | | |
|---|---|---|---|
| **L-03** | Low | The unused-mint burn in addLiquidity retires fees first, so fees become principal | Fixed |
| **L-04** | Low | A redemption removes a whole range before it knows it can repay nothing | Fixed |
| **L-05** | Low | A one-wei rounding shortfall lets a redemption remove part of a minDebt range and repay nothing | Fixed |
| **L-06** | Low | The `rebalance` swap has no caller minimum | Fixed |
| **L-07** | Low | The delegate loss baseline leaves out value the range removal brings in | Fixed |
| **L-08** | Low | The fixed one-day loss window lets a delegate spend twice the budget across a reset | Acknowledged |
| **L-09** | Low | A delegate `rebalance` with no swap and no mint skips the price freshness check | Fixed |
| **I-01** | Info | sqrtPriceFromOracle overflows for a low-decimal asset at a high price | Acknowledged |
| **I-02** | Info | Aerodrome addLiquidity measures tokens used by balance difference across a gauge restake | Acknowledged |
| **I-03** | Info | A pool asset equal to the DEX reward token turns emissions into collateral | Acknowledged |
| **I-04** | Info | addLiquidity and reposition have no slippage limit beyond the pool-price band | Acknowledged |
| **I-05** | Info | eUSD in a range counts at $1, so a single-sided eUSD range is leveraged on the asset | Acknowledged |
| **I-06** | Info | A partial liquidation of a position below 100% can lower its ratio | Acknowledged |

| | | | |
|---|---|---|---|
| **I-07** | Info | A liquidator earns no bonus when the pool trades above the range | Acknowledged |
| **I-08** | Info | The operator mint pause does not stop redemption | Acknowledged |
| **I-09** | Info | closePrincipalOnly settles fees in asset at the bare oracle price | Acknowledged |
| **I-10** | Info | The unused-mint burn in addLiquidity emits no event | Fixed |
| **I-11** | Info | A pool price band of 100% makes the rebalance swap minimum zero | Fixed |
| **I-12** | Info | `harvest` does not refuse a reward token equal to eUSD | Fixed |
| **I-13** | Info | A PegArbitrage sale sized to the exact ceiling room reverts | Acknowledged |
| **I-14** | Info | `_mintFromPsm` has an unreachable rounding branch | Acknowledged |
| **I-15** | Info | PSM debt below one asset unit cannot be redeemed | Acknowledged |

## Status of Earlier Findings

- No earlier report covers the EUSDLiquidityManager. The 01-11-2026 report's acknowledged item on a blocklisted owner's liquidation refund in EUSDManager has the same cause as L-02 here.
- The 01-11-2026 report's I-03 (no redemption fee in EUSDManager) and its accepted in-session gap under M-01 are unchanged in code. The PSM supplies eUSD at exactly $1 up to its ceilings, which removes the pool premium a redeemer used to pay; see M-07.

## Properties Attacked and Held

**EUSDLiquidityManager:**

- eUSD supply equals totalDebt, which equals the sum of position debts, on every path (mint, repay, close, both liquidation kinds, redemption, rebalance, the unused-mint burn and fee accrual).
- Every position's debt is zero or at least minDebt after every call.
- feesAccrued never exceeds debt.
- Valuation at the oracle price is a lower bound for the range at any pool price, so moving the pool can only lower a position's value, never raise it.
- Minted eUSD goes only into the position's vault, never to a user wallet.
- After the M-01 fix, a partial liquidation of a position at or above 100% never lowers its ratio.
- After the L-01 fix, the liquidator's take is cleared × bonus whatever the pool split.
- Redemption: the payout never exceeds the eUSD burned (rounding is against the redeemer), never exceeds the vault's asset, and raises the owner's ratio. Pushing the pool down to make a position eligible costs the attacker more than the payout. Naming an owner twice, naming owners with no debt, and redeeming one wei are all safe.
- The staked and unstaked fee indexes stay consistent across stake switches.
- After the H-02 fix, `rebalance` adds the new range only at a pool price inside the band, after its own swap.
- After the M-03 and L-07 fixes, a delegate's equity loss at the oracle price is charged to the vault's daily budget, measured after the old range is removed; eUSD paid to the owner counts as kept.
- After the M-04 fix, `liquidate`, `collateralRatioBps` and `isLiquidatable` all net idle eUSD against the debt first.

**FablesAdapter / AerodromeAdapter:**

- Only the manager can move principal; the owner can only sweep tokens other than the asset and eUSD. The delegate can only call `rebalance` and `harvest`.
- After the H-01 fix, a full removal burns the Aerodrome NFT's real liquidity, so donated liquidity cannot block a close or a liquidation.

- The Fables pool price read from the PoolManager storage slot matches the live contract.
- Vault swaps route only through pools the Fables registry lists; Fables hooks refuse to initialize a pool their admin has not configured.

**EUSDPsm:**

- For every listed stablecoin, balance × scale ≥ debt across mint, redeem and withdrawSurplus, with fees and rounding in both directions; totalDebt equals the sum of the per-asset debts.
- Fees cannot be avoided with small or split amounts; fee-on-transfer tokens are refused.
- The PSM mints to and burns from the caller only; withdrawSurplus pays only the surplus above debt, only to the treasury.
- The reentrancy guard uses a namespaced slot and works behind the UUPS proxy; the implementation is locked and upgrades need ADMIN.

**PegArbitrage:**

- No caller without a role can trade or move funds; callbacks are accepted only from the PoolManager during the helper's own v4 trade and from the pool of its own v3 trade.
- With an honest operator and a genuine pool, the swap direction, the delta signs and the settle order are correct on both paths, and every PSM mint and redeem inside a trade is fully paid. The PSM reserve is never reachable.
- withdraw pays only the treasury.

# Issue H-01: Donated liquidity on an unstaked Aerodrome vault NFT makes the whole debt unliquidatable RESOLVED

Severity: High

## Vulnerability Detail

The Robinhood Chain Aerodrome position manager restricts `increaseLiquidity` only while the NFT is in the gauge. Anyone can therefore add liquidity to an unstaked vault NFT. The vault tracked only the liquidity it added itself, and a full removal removed only that amount, so `burn` reverted because liquidity remained.

The owner could then shrink the tracked liquidity to 1 with `removeLiquidity`. Once the range has filled to all asset, this does not change the debt. From then on, every liquidation computes `share = ceil(1 × cleared / debt) = 1 = held` and takes the full-removal path, which reverts.

Example: an eUSD-only range sits below the price with 3,000 eUSD of debt. The price falls through the range, the owner removes `held − 1`, and someone donates dust liquidity. With the oracle far below the range (ratio 49%), `liquidate(max)`, `liquidate(debt / 2)` and a small liquidation all revert.

## Impact

The owner holds a free put on the asset: the debt can never be liquidated, so the protocol carries the bad debt, bounded only by the debt ceiling. A third party can also block any unstaked position's close and reposition.

## Resolution

A full removal now reads the NFT's actual liquidity from `positions(id)` and removes all of it. Donated tokens land in the vault as idle balance. Regression tests: `test_donatedLiquidity_closeStillBurns`, `test_donatedLiquidity_heldOne_liquidationsWork` and the fork test `test_fork_aero_donatedLiquidity_liquidationsWork`. The mock position manager no longer gates `increaseLiquidity`.

## Issue H-02: `rebalance` checks the pool price band before its own swap, so the new range can open at a price the swap moved RESOLVED

Severity: High

### Vulnerability Detail

`rebalance` checked the pool price against the band around the oracle price once, in `_snapshot`, before anything moved. It then removed the old range and, if asked, swapped in the vault's own pool. The swap had no price limit, and `_add` deposited the new range at whatever price the swap left. `_checkHealth` reads no pool price and checks nothing when the debt is zero.

Example with an illustrative 2% band: the pool sits at 100. A swap whose fills average 98 (inside the band) leaves the pool near 96, and the new range opens there. A back-run then buys the range's asset below the oracle price. In a thin pool a test setup lost about 17% of the eUSD added.

### Impact

An honest call can leave the new range well outside the band, where it is arbitraged at the owner's cost. A delegate acting against the owner can push the pool far with the last part of its own swap and buy the owner's asset cheaply in the same call.

### Resolution

Fixed in 0771f91. `rebalance` checks the band again after the swap, before `_add`, so a swap that moves the pool out of the band reverts. Regression test: `test_rebalance_swapMovesPoolOutOfBand_reverts`.

## Issue M-01: A partial liquidation can take more than its share of the collateral and leave bad debt RESOLVED

Severity: Medium

### Vulnerability Detail

`_payLiquidator` capped the asset seized at the vault's whole asset balance, not at the share of the position being cleared.

Example with a 5% bonus: debt is 1,000,000 and the vault's asset is worth 1,020,000. A partial liquidation of 971,000 takes all of the asset and leaves 28,600 of debt with no collateral. A full liquidation of the same position leaves no bad debt.

### Impact

A liquidator can turn a solvent position (ratio between 100% and 100% + bonus) into bad debt.

### Resolution

A partial liquidation now takes at most `value × cleared / debt` in total (vault eUSD burned plus asset seized), the same pro-rata cap EUSDManager uses. Regression test: `test_liquidate_partial_belowOnePlusBonus_paysOnlyShareOfValue`.

## Issue M-02: Redemption pays the oracle price with no buffer, so redeemers keep the oracle's lag behind the market RESOLVED

Severity: Medium

### Vulnerability Detail

A redeemer burned eUSD and received asset at the oracle price with no fee. A redemption-only price age check was meant to keep that price current. The oracle reports any Chainlink answer younger than its fresh window as current (`block.timestamp`), and Chainlink stock feeds update on a deviation threshold. So the check did not limit how far the price could trail the market.

Example with an illustrative 0.5% deviation threshold: the feed shows 100.00 and the market trades at 100.45. A redeemer sells asset into the pool for eUSD, redeems 10,000 eUSD for 100 units of asset, and sells them for 10,045. Positions whose range sits above the price (all asset) are always eligible, so no sell-off or depeg is needed.

### Impact

Owners of eligible positions lose the oracle–market gap on every redemption, up to the feed's deviation threshold during market hours and more when off-hours prices are held inside their band.

### Resolution

Redemption now pays `repaid × (1 − redeemFeeBps) / price`. The fee stays in the owner's vault and is set by ADMIN; it should be at least the feed's deviation threshold. The redemption-only price age was removed, and redemption uses `mintPriceMaxAge`. A zero `redeemMinAssetShareBps` turns redemption off. Regression test: `test_redeem_feeStaysWithOwner`.

## Issue M-03: A delegate can repeat `rebalance` with no limit on the owner's total loss RESOLVED

Severity: Medium

### Vulnerability Detail

Each `rebalance` swap was bounded only by `minOut = fair × (1 − poolPriceBandBps)`, and nothing limited how many calls or swaps a delegate made. `_checkHealth` only stops a fall below MCR, and a swap from asset to eUSD that repays debt removes even that floor. The delegate can be the counterparty: it can sandwich the vault's own pool, or place a narrow one-sided range at the band edge and trade through it, which needs no swap and also works on Aerodrome.

Example with an illustrative 1% band: a vault with no debt holds $200,000. About 70 swaps in one transaction move about $100,000 to the delegate's side. This contradicts the interface line that a delegate can never move tokens out of the vault, because value leaves through trades.

### Impact

A rogue or stolen delegate key can take an owner's equity down to MCR, or all of it at zero debt. One agent key can serve many vaults.

### Resolution

Fixed in 0771f91. The owner sets a loss budget with `setDelegate(delegate, lossBudgetBps)`. Each delegate `rebalance` charges its equity loss at the oracle price (eUSD paid to the owner counts as kept) to a one-day window on the vault and reverts past the budget. Regression tests: `test_rebalance_delegateLossBudget_capsDailyLoss`, `test_rebalance_delegateZeroBudget_noLossAllowed`, `test_rebalance_delegatePayoutToOwner_isNotALoss`.

## Issue M-04: `liquidate` counts harvested idle eUSD on both sides, so a healthy position can be liquidated RESOLVED

Severity: Medium

### Vulnerability Detail

`FablesAdapter.harvest` can leave the eUSD it buys idle in the vault. Every manager path burns idle eUSD against the debt (`_burnIdle`), but `harvest` is a vault call and does not. `liquidate` compared value with debt gross, so idle eUSD counted on both sides, and a position above the threshold after netting could be below it gross.

Example with an illustrative 110% threshold and 5% bonus: debt 10,000, range worth 8,900, idle eUSD 2,000. Gross the ratio is 109% (liquidatable); netted it is 8,900 / 8,000 = 111% (healthy). A partial `liquidate(2,000)` burns the vault's eUSD and pays the liquidator $100 of asset for no eUSD. A full liquidation nets the liquidator $500 and takes the owner's equity from 900 to 400.

### Impact

Owners of healthy positions lose the liquidation bonus on the idle eUSD amount. The size depends on how much harvested eUSD sits idle until the next rebalance or add.

### Resolution

Fixed in 98d64ef. `liquidate` calls `_burnIdle` right after `_accrue`, and `collateralRatioBps` and `isLiquidatable` net idle eUSD the same way, so keepers see what `liquidate` sees. Regression test: `test_liquidate_burnsIdleEusdBeforeRatio`.

## Issue M-05: The OPERATOR can send the PegArbitrage float to a pool it controls ACKNOWLEDGED

Severity: Medium

### Vulnerability Detail

`arbitrage` (v4) accepts any pool key with eUSD in it, and its only loss check is `profit < minProfit` with a `minProfit` the same caller picks. An OPERATOR can open its own eUSD pool at a far price with narrow liquidity and call `arbitrage` with `minProfit` set to minus the float; the swap pays the float into that pool. On the real pool, the OPERATOR can push the price the wrong way, call `arbitrage` in that direction with a negative `minProfit`, and trade back.

`arbitrageV3` accepts any contract as `pool`, and `uniswapV3SwapCallback` trusts that contract. It reads the stablecoin from the pool-returned `data` rather than from the helper's own state, and it accepts more than one callback per trade. A fake pool can therefore name a different listed stablecoin than the one `arbitrageV3` measures, so every listed stablecoin's float leaves with a measured profit of zero, even with `minProfit ≥ 0`.

Example: the helper holds 10,000 units of a listed stablecoin. The OPERATOR's pool prices eUSD at 10,000 units, and `arbitrage(key, sellEusd = false, limit, 10,000e6, −10,000e6)` pays 10,000 for a little eUSD; the check `−9,999e6 ≥ −10,000e6` passes.

### Impact

A holder of the OPERATOR key (the keeper key or the Safe) can take the helper's whole balance. The loss is bounded by what PegArbitrage holds; every PSM mint and redeem inside these paths is fully paid, so the PSM reserve and eUSD backing are not reachable.

### Resolution

Acknowledged. The helper holds only a small float, profits are swept to the treasury with `withdraw`, and the keeper trades only pools from its ADMIN-reviewed config with a bounded `minProfit` and a daily loss limit. A fix (an ADMIN pool list with a

per-pool loss floor, and the v3 stablecoin kept in contract state with a once-only callback) can ship in a redeployed helper, which holds no state beyond its float.

## Issue M-06: eUSD minted with a depegged stablecoin redeems a sound one ACKNOWLEDGED

Severity: Medium

### Vulnerability Detail

`redeem` checks only the debt of the stablecoin the caller names, and eUSD is fungible. eUSD minted against one listed stablecoin can therefore take another out of the PSM. `mint` has no price check.

Example with fees of 0: stablecoin B trades at $0.97 and stablecoin A has 100,000 of debt. A caller buys 100,000 B for $97,000, mints 100,000 eUSD (inside B's ceiling), and redeems 100,000 A, for about $3,000 profit. The PSM then holds B where it held A. Each book stays solvent in units (balance × scale ≥ debt) but not in value.

### Impact

A depeg of one listed stablecoin moves its loss onto the PSM, up to that stablecoin's debt ceiling. This is the standard trade-off of a multi-asset PSM.

### Resolution

Acknowledged. A weaker stablecoin is listed with a small ceiling, and OPERATOR stops its minting with `setMintPaused(asset, true)` when its market price leaves a band.

## Issue M-07: A zero-fee PSM mint funds EUSDManager redemptions at a lagging oracle price ACKNOWLEDGED

Severity: Medium

### Vulnerability Detail

With a mint fee of 0, the PSM gives exactly 1 eUSD per $1 of a listed stablecoin with no slippage, up to the ceiling room. `EUSDManager.redeem` pays collateral at `_freshPrice`, which accepts a price up to `mintPriceMaxAge` old, with no fee. Chainlink stock answers update on a deviation threshold. So one transaction can mint at the PSM and redeem at a direct manager whenever the oracle is below the market.

Example with an illustrative 0.4% gap: redeeming 500,000 eUSD earns about $2,000 per round. The room refills whenever eUSD is redeemed at the PSM, so the round repeats. Before the PSM, a redeemer had to buy eUSD in a thin pool above $1, and that premium worked as a fee.

### Impact

Head-of-list borrowers in EUSDManager lose the oracle–market gap on the amount redeemed, up to the feed's deviation threshold per round during market hours and more off-hours. Bounded by the PSM ceiling room and the managers' debt.

### Resolution

Acknowledged. A PSM mint fee would let eUSD trade above $1, which defeats the PSM's purpose. A redemption fee in EUSDManager, as the EUSDLiquidityManager has (M-02), is the fix to add with the next manager upgrade.

## Issue L-01: A liquidator can move the pool price to earn the bonus on the owner's eUSD RESOLVED

Severity: Low

### Vulnerability Detail

Liquidation removes the cleared share of the range at the live pool price, with no band (by design, so a lagging pool cannot block liquidations). The bonus was paid on `repaid = cleared − vault eUSD`. Pushing the pool below the range turns the removed eUSD into asset, which raises `repaid` and the asset seized.

### Impact

The owner loses up to bonus × the removed share's eUSD, less the attacker's swap costs.

### Resolution

The bonus is now paid on `cleared`: `seized = (repaid + cleared × bonus) / price`. The liquidator's take no longer depends on the pool split. Regression test: `test_liquidate_bonusOnCleared_sameWhateverThePoolSplit`.

## Issue L-02: A blocklisted owner makes the final full liquidation revert ACKNOWLEDGED

Severity: Low

### Vulnerability Detail

A full liquidation sends the asset refund to the owner in the liquidator's call. If the asset token blocks the owner, the transfer reverts. The last `minDebt` of a position can only be cleared by a full liquidation.

### Impact

At most about `minDebt` per position stays unliquidatable while the owner is blocked.

### Resolution

Acknowledged, consistent with EUSDManager. No listed asset is known to block holders.

## Issue L-03: The unused-mint burn in addLiquidity retires fees first, so fees become principal RESOLVED

Severity: Low

### Vulnerability Detail

`addLiquidity` mints up to `eusdMax` into the vault and burns what the range did not use. The burn went through `_reduceDebt`, which retires accrued fees first. The owner's unpaid fees were cleared and the same amount stayed as principal.

### Impact

Fees turned into principal that earns further fees, and `closePrincipalOnly` could no longer settle them from asset.

### Resolution

The unused-mint burn now reduces principal only and keeps the `minDebt` rule. Regression tests: `test_add_unusedMint_keepsFeesAccrued`, `test_add_unusedMint_keepsMinDebt`.

## Issue L-04: A redemption removes a whole range before it knows it can repay nothing RESOLVED

Severity: Low

### Vulnerability Detail

`_redeemFrom` set `repaid` from the request, removed range liquidity to free that much asset, and only then clamped `repaid` again for `minDebt`. When the vault's asset was worth less than the request and the remaining gap was below `minDebt`, the second clamp returned zero or `debt − minDebt`.

Example (illustrative `minDebt` of 100): a position owes 100 and its range holds asset worth 97. A redemption of 101 against it and a second owner removes all of its range, repays 0 for it, and redeems from the second owner, so the call does not revert.

### Impact

Anyone can pull a near-`minDebt` position's range out of the pool and gauge for the cost of gas. The owner keeps the value but loses swap fees and emissions until they add liquidity again.

### Resolution

`repaid` is now capped by the eUSD the vault's whole asset covers before any range is removed, and a position that can repay nothing is skipped untouched. Regression test: `test_redeem_belowDebt_atMinDebt_keepsRange`.

## Issue L-05: A one-wei rounding shortfall lets a redemption remove part of a minDebt range and repay nothing RESOLVED

Severity: Low

### Vulnerability Detail

`_redeemCheck` sizes the range's asset with `holdings(poolPrice())`. The pool's sqrt price is converted to an 18-decimal price and back, and both steps round down. When the asset is currency0 and the pool is inside the range, the estimate can exceed what the pool returns by one wei. `_freeAsset` then removes a share of the range and gets one wei less than `seized`. The fallback `_redeemable(debt, _redeemCover(idle))` sees a gap below `minDebt` and, for a position at exactly `minDebt`, returns zero.

Exact-integer replay with test-fixture parameters: 5.7% of the range is removed, nothing is burned, nothing is paid. The redeemer names a second owner so the call does not revert.

### Impact

Grief only. The removed share stays in the vault as idle asset and idle eUSD, so the owner loses nothing but the fee income on that share. It happens once per position: a second call redeems normally.

### Resolution

Fixed in aba9e7e. After `_freeAsset`, `_redeemFrom` now caps `seized` at the asset that is idle and keeps `repaid`, so the redeemer absorbs the rounding dust (at most a few wei). The `redeem` NatSpec says so. Test: `testFuzz_redeem_atMinDebt_poolRoundingStillRepays` puts the pool just above the range's lower tick, redeems a position at `minDebt`, and checks it is fully repaid and the payout is short by at most 2 wei. The test fails on 5a834c0.

## Issue L-06: The `rebalance` swap has no caller minimum

RESOLVED

Severity: Low

### Vulnerability Detail

`RebalanceParams` had no swap minimum. The only floor on the swap output was the oracle price less the band, so a trader who moved the pool around the call could keep up to the band on the swap amount.

### Impact

An owner or delegate swap could fill up to the band below fair value. Bounded by the band on the swap amount per call.

### Resolution

Fixed in 0771f91. `RebalanceParams.swapMinOut` sets the caller's minimum, and the swap takes the higher of it and the oracle floor. A deadline was not added. Regression test: `test_rebalance_swapMinOut_reverts`.

## Issue L-07: The delegate loss baseline leaves out value the range removal brings in RESOLVED

Severity: Low

### Vulnerability Detail

The first loss-budget version took the delegate's starting equity from the snapshot, before `_removeAll`. Value that the removal brings into the vault was missing from that baseline: Aerodrome swap fees collected on removal, and the gap between the oracle lower bound and the range's real value when the pool sits off the oracle. A delegate working with a trader could take that value plus the budget.

### Impact

On Aerodrome vaults, uncollected swap fees could leave the vault without being charged. The lower-bound gap is paid only when the pool lags the oracle on its own. Fables removals collect principal only.

### Resolution

Fixed in a9be8d3. The delegate baseline is read again with `_value` after `_removeAll`, before the swap. Regression test: `test_rebalance_delegateLossCountsCollectedFees`.

## Issue L-08: The fixed one-day loss window lets a delegate spend twice the budget across a reset ACKNOWLEDGED

Severity: **Low**

### Vulnerability Detail

`DelegateLoss.charge` resets the used budget when a full day has passed since the window started. A delegate can spend the whole budget just before the window ends and the whole budget again just after.

### Impact

Up to twice the daily budget can be lost within seconds, once per window.

### Resolution

Acknowledged. The fixed window keeps the code small, and owners size the budget knowing the bound is twice its value.

## Issue L-09: A delegate `rebalance` with no swap and no mint skips the price freshness check RESOLVED

Severity: Low

### Vulnerability Detail

`_snapshot` reads the oracle price with no age check. The freshness check ran only in `_swap` and in `_checkHealth` when the debt grows. A delegate `rebalance` with no swap and no mint could place a narrow one-sided range at a stale price, and the delegate could then trade through it after the call. The loss was measured at the stale price, so the budget recorded almost nothing.

### Impact

Only when the in-house signer stops and the oracle falls back to an older Chainlink answer, and the market moves more than the band away from it.

### Resolution

Fixed in 98d64ef. `rebalance` requires a fresh price when the caller is the delegate. Regression test: `test_rebalance_delegateStalePrice_reverts`.

## Issue I-01: sqrtPriceFromOracle overflows for a low-decimal asset at a high price ACKNOWLEDGED

Severity: Info

### Vulnerability Detail

`Math.mulDiv(price, Q192, assetScale)` overflows when the price per raw asset unit is at least 2^64: above about $18.4 for a 0-decimal asset, $1,844 for 2 decimals and $18.4M for 6 decimals.

### Impact

Valuation of that pool reverts. 18-decimal assets are never affected.

### Resolution

Acknowledged. Do not list assets below 6 decimals.

## Issue I-02: Aerodrome addLiquidity measures tokens used by balance difference across a gauge restake ACKNOWLEDGED

Severity: Info

### Vulnerability Detail

For a staked NFT, `addLiquidity` measures the asset used as a balance difference that spans `gauge.withdraw` and `gauge.deposit`. If the gauge paid the pool asset into the vault, the subtraction would underflow.

### Impact

Only a pool whose asset is the gauge's reward token is affected, and only the caller's call reverts.

### Resolution

Acknowledged. No such pool is planned.

## Issue I-03: A pool asset equal to the DEX reward token turns emissions into collateral ACKNOWLEDGED

Severity: Info

### Vulnerability Detail

In a pool whose asset is FABLES (or the Aerodrome reward token), claimed emissions are the protected asset. `FablesAdapter.claimEmissions` then reverts in `_sweep` with `ProtectedToken`.

### Impact

The emissions stay in the vault as collateral and leave on close.

### Resolution

Acknowledged. No such pool is planned.

## Issue I-04: addLiquidity and reposition have no slippage limit beyond the pool-price band ACKNOWLEDGED

Severity: Info

### Vulnerability Detail

The only guard on the pool price at add and reposition time is the manager's band around the oracle price, plus the caller's maximum amounts.

### Impact

A caller can get a worse fill inside the band. Someone holding the pool out of band can block an add for one block.

### Resolution

Acknowledged. The band is the intended guard. `rebalance`, which replaced `reposition`, also checks the band after its own swap (H-02) and takes a caller swap minimum (L-06).

## Issue I-05: eUSD in a range counts at $1, so a single-sided eUSD range is leveraged on the asset ACKNOWLEDGED

Severity: Info

### Vulnerability Detail

A range below the price holds only eUSD, valued at $1. As the price falls through it, the range fills with asset bought with that eUSD, so the position is long the asset with leverage of about 1 / (MCR − 1).

### Impact

A parameter note for low-MCR managers, not a code defect.

### Resolution

Acknowledged.

## Issue I-06: A partial liquidation of a position below 100% can lower its ratio ACKNOWLEDGED

Severity: Info

### Vulnerability Detail

The pro-rata cap from M-01 bounds the asset seized but not the vault eUSD burned against the cleared debt. For a position already below 100% that holds idle eUSD, the burn can exceed the cap.

Example: debt 100, idle eUSD 90, idle asset worth 5 (ratio 95%). `liquidate(50)` burns 50 vault eUSD and seizes nothing; `_burnIdle` burns the other 40. Debt is 10, value is 5, ratio 50%.

### Impact

None in value: owner equity, bad debt and the supply invariant are unchanged, and the liquidator gains nothing. The interface's statement that the remaining ratio never falls holds only at or above 100%.

### Resolution

Acknowledged.

## Issue I-07: A liquidator earns no bonus when the pool trades above the range ACKNOWLEDGED

Severity: Info

### Vulnerability Detail

When the pool is above the range, the removed share is all eUSD, `repaid` is zero and the asset cap leaves no bonus. The debt is still cleared one for one from the vault's eUSD.

### Impact

Liquidators have no incentive in this state, but solvency improves with every such liquidation.

### Resolution

Acknowledged. Someone who pushes the pool above the range just before a liquidation leaves the liquidator little or no bonus, at their own cost. The `liquidate` NatSpec now says that liquidators should simulate first or send privately (ae2a1f5).

## Issue I-08: The operator mint pause does not stop redemption ACKNOWLEDGED

Severity: Info

### Vulnerability Detail

`withdraw` checks `mintPaused` before it releases asset at the oracle price; `redeem` does not. Only ADMIN can stop redemption, by setting `redeemMinAssetShareBps` to 0.

### Impact

During an oracle incident, redemptions continue at a suspect price (within the oracle's band) until ADMIN acts. Every redemption still raises the owner's ratio.

### Resolution

Acknowledged. The mint pause is for minting; ADMIN stops redemption with `setRedeemParams(0, feeBps)`. Every redemption raises the owner's ratio, so a short delay does not put a position at risk.

## Issue I-09: closePrincipalOnly settles fees in asset at the bare oracle price ACKNOWLEDGED

Severity: Info

### Vulnerability Detail

`closePrincipalOnly` pays the accrued fees in asset at the oracle price, rounded up by one unit. The owner chooses per call between paying fees in eUSD (`close`) and in asset, so they can pick asset when the oracle is above the market.

### Impact

The treasury receives asset worth up to `feesAccrued × oracle gap` less than the fees. Bounded by the position's own fees.

### Resolution

Acknowledged. The gap is bounded by the position's own fees and the oracle's band, and the path exists so an owner can close without buying eUSD.

## Issue I-10: The unused-mint burn in addLiquidity emits no event RESOLVED

Severity: Info

### Vulnerability Detail

`_add` emits `EUSDMinted` with the debt before the unused burn, and `_burnUnminted` lowers the debt without an event. `_burnIdle` emits `DebtRepaid` for the same kind of burn.

### Impact

An indexer that rebuilds debt from events overcounts it. On-chain state is correct.

### Resolution

Fixed in aba9e7e. `_burnUnminted` emits `DebtRepaid(poolId, owner, vault, amount, newDebt)`, the same event `_burnIdle` emits. Test: `test_add_unusedMint_emitsDebtRepaid` (fails on 5a834c0).

## Issue I-11: A pool price band of 100% makes the rebalance swap minimum zero RESOLVED

Severity: Info

### Vulnerability Detail

`initialize` and `setPoolPriceBand` accepted `poolPriceBandBps == BPS`. The swap floor is `fair × (BPS − poolPriceBandBps) / BPS`, which is then zero.

### Impact

An ADMIN setting would remove the swap floor. No effect at the configured bands.

### Resolution

Fixed in 0771f91. The band must be below 100%. Regression test: `test_setPoolPriceBand`.

## Issue I-12: `harvest` does not refuse a reward token equal to eUSD RESOLVED

Severity: Info

### Vulnerability Detail

`FablesAdapter.harvest` refused a reward token equal to the asset but not one equal to eUSD; `sweep` refuses both.

### Impact

Unreachable with the live reward token.

### Resolution

Fixed in 0771f91. `harvest` also refuses a reward token equal to eUSD. Regression test: `test_harvest_fablesIsEusd_reverts`.

## Issue I-13: A PegArbitrage sale sized to the exact ceiling room reverts ACKNOWLEDGED

Severity: Info

### Vulnerability Detail

`_mintFromPsm` rounds the stablecoin paid in up, so the PSM can mint up to one stablecoin unit more eUSD than the pool takes, and that counts against the ceiling. A sale with `maxIn` equal to the remaining room reverts with `DebtCeilingExceeded`.

### Impact

No funds move; the correction is delayed until a smaller trade.

### Resolution

Acknowledged. The keeper sizes sales below the room.

## Issue I-14: `_mintFromPsm` has an unreachable rounding branch ACKNOWLEDGED

Severity: Info

### Vulnerability Detail

In `_mintFromPsm`, the line `if (out < eusdOwed) ++assetIn;` can never run: rounding the input up already gives at least `eusdOwed` for every decimals and fee setting.

### Impact

None; a small amount of dead code.

### Resolution

Acknowledged.

## Issue I-15: PSM debt below one asset unit cannot be redeemed ACKNOWLEDGED

Severity: Info

### Vulnerability Detail

`redeem` reverts with `ZeroAmount` when the eUSD amount is worth less than one unit of the stablecoin. Debt below one unit stays booked, and one unit of reserve stays covering it.

### Impact

At most one unit of each stablecoin (for a 6-decimal token, $0.000001).

### Resolution

Acknowledged.